

MySQL & Database Fundamentals

- What is MySQL? 🤖
 - It's a **Relational Database Management System (RDBMS)**. Think of it like a super-organized digital filing cabinet! 🗄️
 - Organizes data into **tables** (like spreadsheets) with **columns** (data types) and **rows** (individual records).

- MySQL & Linux Servers 🐧
 - MySQL software typically lives and runs on a **Linux Server**. Linux is its cozy home! 🏠
 - **Analogy:**
 - Your **Linux Server** is the whole 🏠 **house**.
 - The **DB Server** (MySQL instance) is a specific 🚪 **room** in that house.
 - A **Schema** (or **Database** in MySQL terms) is like a **rack** 📁 holding many **tables**.
 - A single DB server can manage many **transactions** and **users** at once. 📊 👥
- **Checking MySQL Connectivity:**
 - `telnet mysql.skptech.site 3306` 📞 - This command is your quick check to see if MySQL is awake and listening on its default port (3306). If it connects, you're golden! ✨

Understanding Proxies: Your Network Middlemen!

Proxies are like helpful intermediaries for your network traffic.

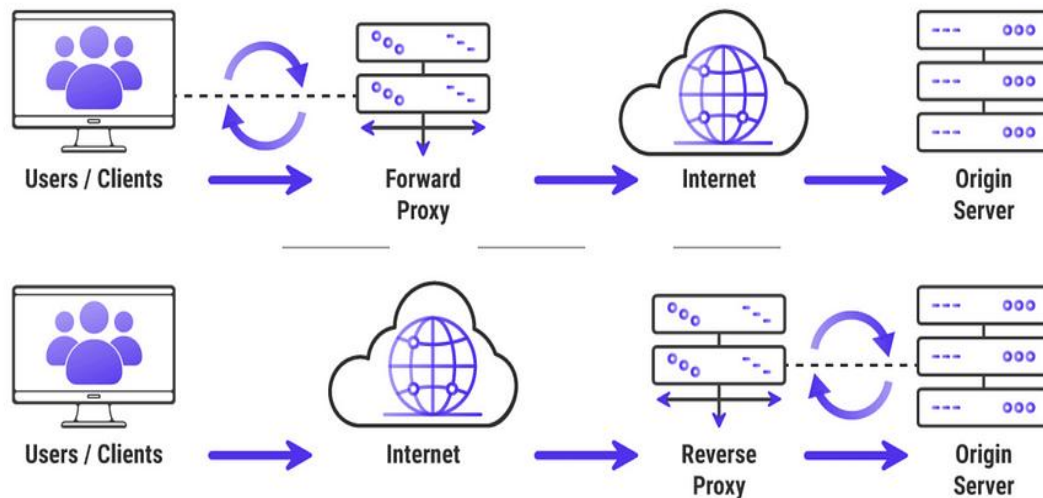
Forward Proxy Client's Best Friend

- **What it does:** Hides the **client's identity** from the server. The server only "sees" the proxy. 
- **Flow:** Your Actual Client  Forward Proxy  Server
- **Key Uses:**
 1. **Content Filtering:** Blocking naughty websites! 
 2. **Traffic Monitoring:** Keeping an eye on web activity. 
 3. **Anonymous Client:** Surf the web privately. 
 4. **Geo Restrictions Bypass:** Accessing content from anywhere.  
 5. **Caching:** Speeding things up by storing frequently accessed data. 

Reverse Proxy Server's Shield & Organizer

- **What it does:** Hides the **server's identity** from the client. The client only "sees" the reverse proxy. 
- **Nginx Powerhouse:** Nginx is a superstar that can be a **reverse proxy**, **load balancer**, and **web server**. It's incredibly versatile! 
- **Flow:** Client  Reverse Proxy (Frontend)  Backend Server(s)

Forward Proxy vs Reverse Proxy



- **Key Uses:**

1. **Server Anonymity:** Keeps your backend servers safe and hidden. 🛡️
2. **Security:** Adds a layer of defense against direct attacks. 🛡️
3. **Load Balancing:** Distributes incoming requests across multiple backend servers to prevent overload. Think of it as a traffic cop! 🚦
4. **SSL Termination:** Handles the heavy lifting of decrypting secure (HTTPS) traffic, offloading work from your backend servers. 🛡️ ➡️ 💻

- **Checking Backend Servers:**

- telnet backend.skptech.site 8080 📞 - Test connectivity to your backend application.
- systemctl status backend - Check if your application service is running on the backend. ✅
- Check **logs** in /var/log for any issues! 📄

HTTP Status Codes: Server's Response Language

When you request something from a web server, it sends back a three-digit status code.

- **1XX: Informational** (e.g., 100 Continue) 
- **2XX: Success!** (e.g., 200 OK, 201 Created) 
- **3XX: Redirection** (e.g., 301 Moved Permanently) 
- **Errors!** 
 - **4XX: Client Side Error** (You messed up, sorry!) 
 - 400 Bad Request
 - 401 Unauthorized
 - 403 Forbidden
 - 404 Not Found (The classic!) 
 - **5XX: Server Side Error** (Oops, the server had a problem!) 
 - 500 Internal Server Error
 - 502 Bad Gateway
 - 503 Service Unavailable

CRUD Operations: The Database Basics

These are the four fundamental actions you perform on data in a database.

- **C - CREATE (INSERT):** Adding new data. 
 - Example: `INSERT INTO users (name) VALUES ('Alice');`

- **R - READ (SELECT):** Getting data out. 🤖
 - Example: `SELECT * FROM products WHERE price < 100;`
 - **U - UPDATE:** Changing existing data. ✎
 - Example: `UPDATE inventory SET stock = 50 WHERE item = 'Widget';`
 - **D - DELETE:** Removing data. 🗑️
 - Example: `DELETE FROM logs WHERE date < '2023-01-01';`
-

Beyond the Basics: Applications & Concepts

- **Reports:** Generating daily, weekly, monthly, quarterly insights from your data is crucial for business decisions. 📊
- **Data Types:** Examples like bank-records, user, notifications, and ai/ml show the diverse kinds of information stored in databases. 📄
- **SaaS (Software as a Service):** This entire infrastructure (Linux, MySQL, proxies, web apps) is often the backbone of a SaaS offering, where software is delivered over the internet.

